

Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification – Campus Placement & Internship Pipeline

Team Member	SRN	Team Member	SRN
Gaurav Girish	PES1UG24AM103	Sunay R Hegde	PES1UG24AM903
Dilipan Y	PES1UG24AM092	Dhruv Rajagopalan	PES1UG24AM086

Architecture Selection: “We chose Microservices Architecture for the Campus Placement & Internship Pipeline System.”

1. Architectural Choice

Microservices Architecture. The system is split into five small, independently deployable services, one per business capability: Profile & Resume, Eligibility, Interview, Offer and Application Status. Each service owns its own database. The Web Portal reaches them only through an **API Gateway**, services call each other through REST interfaces (ISkillProfile, IEligibilityCheck, ISelectionResult), and status changes are shared asynchronously through a **Message Broker**, as shown in our component diagram.

2. Two Reasons for the Choice

Reason 1 – Placement drives create sudden, uneven load. When a drive opens, hundreds of students upload resumes and must be screened within a short window, while the interview and offer modules see light, steady use. With microservices we can run extra instances of only the Profile & Resume and Eligibility services to meet NFR-002 (95% of screening results within 5 seconds at 100 concurrent requests), instead of scaling the whole application as a Layered or Client-Server system would require.

Reason 2 – The pipeline consists of separate modules that change and fail independently. Resume parsing, eligibility screening, interviews, offers and status tracking map one-to-one to our epics. Eligibility rules change for every company drive, and resume parsing (our riskiest story) may need a different parser or technology. Each service can be updated and redeployed alone. A failure is also contained: if the parser crashes on a malformed PDF, recruiters can still record interview outcomes and students can still view their application status.

3. Security Advantage

A single, controlled entry point and isolated data. Every request passes through the API Gateway, which authenticates the user (JWT) and enforces role-based access before routing. For example, only a Placement Officer can configure eligibility rules or create offers, and a student can see only their own records. Because of database-per-service, sensitive academic records and offer letters live only in the Profile and Offer databases, encrypted with AES-256 (NFR-001) and reachable only through those services' APIs. A compromise of one service therefore does not expose all student data.

4. Performance Benefit

Independent scaling and asynchronous status updates. The CPU-heavy work (PDF parsing and eligibility checks) runs on its own load-balanced service instances, so it never slows down the other modules. Status changes are pushed as events (EligibilityDecided, OutcomeRecorded, OfferIssued) into the Application Status Service's own read model, so a student's "View Application Status" request is a single fast lookup that does not wait on the other services. This keeps screening within the 5-second target and the dashboard responsive on busy drive days.

Alternatives considered and rejected

- **Layered:** a single deployable unit, so the whole system must be scaled together, and a parser failure can take down the entire portal.
- **Client-Server:** the central server becomes a bottleneck and a single point of failure on drive days.
- **Trade-off accepted:** microservices add operational complexity and eventual consistency; the API Gateway and Message Broker keep these manageable, and a few seconds of delay in status updates is acceptable.